Project Echo: Error Analysis of the EfficientNetV2 Production Classifier

Deakin University SIT374/SIT764 Capstone Project

Jack Montgomery-Parkes
Deakin University
s223232288@deakin.edu.au

1 Introduction

1.1 Background

Following the architecture benchmarking study conducted earlier in this project, EfficientNetV2 was selected as the production classifier for the Project Echo Engine on the basis of its strong accuracy-to-efficiency profile. That model has since been integrated into the Engine pipeline as a TFLite artefact, with an initial val run reporting 94% top-1 accuracy across a 50-file sample.

That headline number, whilst reassuring, is not enough to direct further development work. A single aggregate accuracy figure tells us nothing about which species the model handles well, where it fails, what kinds of mistakes it makes, or whether its confidence outputs can be trusted to filter results downstream. Without that detail, any attempt to improve the model in the next sprint would be speculative; we would not know where to focus.

1.2 Scope

This report performs a targeted error analysis of the selected EfficientNetV2 model against a properly stratified test set covering all 123 species in the dataset. The intent is to characterise where the model succeeds, where it fails, and why, so the next sprint of development work can proceed with something approaching a clear direction.

The analysis covers four areas: class-wise and species-wise performance, dominant confusion patterns, confidence behaviour on correct versus incorrect predictions, and likely causes of the errors observed. These findings are then translated into a set of workable recommendations for Sprint 3.

2 Methodology

2.1 Test Set Construction

The initial 50-file validation run that reported 94% accuracy drew its sample largely from alphabetically-early classes, leaving the bulk of the 123-class output space effectively untested. To produce a meaningful error analysis, a properly stratified sample was constructed by drawing up to 20 audio clips per class (`seed=42`), or all available files where a class held fewer than 20. This approach balances statistical power on well-represented classes against the practical reality that several classes in the training data hold fewer than 10 samples in total. The final manifest covered all 123 classes with 2,166 files, against an originally validated baseline of 50.

2.2 Inference Pipeline

We started by fetching and setting up `efficientnetv2_predictor.py` from an earlier sprint 2 task, making sure that the preprocessing pipeline tested here matches what the Engine container uses in production. For each file in the manifest, we record the predicted label, top-5 predictions with confidences, and the rank and confidence assigned to the true label. All inference was conducted using the PyTorch (`.pt`) model with a fair assumption made, that it is numerically equivalent to the deployed TFLite variant. PyTorch was selected for it's speed and ease of use with MPS architecture, however, a future spot-check across a sample of files would verify this at scale using the original Torch backbone.

2.3 Evaluation Metrics

Top-1 and top-5 accuracy are reported alongside per-class precision, recall, and F1. Macro-averaged F1 is given priority over micro accuracy where it matters, specifically with regards to class imbalance. Mean prediction confidence is reported separately for correct and incorrect predictions, with a precision-versus-threshold sweep used to inform deployment-side filtering recommendations. We're also grabbing the expected calibration error in order to measure how trustworthy the model's confidence values are in absolute terms.

3 Results

3.1 Overall Performance

On the stratified 2,166-file test set, the EfficientNetV2 model achieves a top-1 accuracy of 95.8% and a top-5 accuracy of 98.3%. Per-class F1 scores reveal a strong baseline with a localised weakness: 114 of 123 classes score above 0.9, and no class fails entirely.

Table 1: Aggregate performance on the stratified test set.

Metric	**Value**
Top-1 accuracy	0.958
Top-5 accuracy	0.983
Macro precision	0.967
Macro recall	0.957
Macro F1	0.960
Classes with F1 $\geq$ 0.9	114 / 123
Classes with F1 = 0	0 / 123

That macro F1 (0.960) and micro accuracy (0.958) are essentially identical is worth poking at. This is likely a consequence of our stratified test set. By flattening the natural class imbalance, each class contributes roughly equal weight to both metrics so they cannot diverge as much as they would on a more naturally-distributed test set.

3.2 Worst-Performing Classes

Despite a strong overall picture, we have nine classes that sit below 0.9 F1 and could use some future work. Table 2 lists these along with the next ten classes immediately above the threshold, sorted ascending by F1.

Table 2: Twenty lowest-F1 classes.

Class	Support	Precision	Recall	F1
Rhipidura leucophrys	20	0.594	0.950	0.731
Rhipidura albiscapa	20	0.783	0.900	0.837
Acanthorhynchus tenuirostris	20	0.783	0.900	0.837
Anhinga novaehollandiae	9	1.000	0.778	0.875
Fulica atra	14	1.000	0.786	0.880
Colluricincla harmonica	20	0.800	1.000	0.889
Vanellus miles	20	1.000	0.800	0.889
Philemon citreogularis	20	0.800	1.000	0.889
Lalage leucomela	20	0.944	0.850	0.895
`sheowl`	20	0.900	0.900	0.900
Haliastur sphenurus	20	0.900	0.900	0.900
Chenonetta jubata	15	0.875	0.933	0.903
Spilopelia chinensis	10	0.833	1.000	0.909
Falcunculus frontatus	13	1.000	0.846	0.917
Megapodius reinwardt	13	1.000	0.846	0.917
Melithreptus brevirostris	20	1.000	0.850	0.919
Eurystomus orientalis	20	1.000	0.850	0.919
Phaps elegans	20	1.000	0.850	0.919
Acanthiza chrysorrhoa	20	0.947	0.900	0.923
Parvipsitta pusilla	20	0.947	0.900	0.923

The precision-recall imbalance in this table point at two distinct weaknesses in our current setup. Classes such as *Rhipidura leucophrys* and *Colluricincla harmonica* show high recall but low precision. For these classes, the model rarely misses them but predicts them too often elsewhere. On the other hand, *Anhinga novaehollandiae*, *Vanellus miles*, and *Falcunculus frontatus* show high (near perfect) precision but low recall. For these, the model is correct when it picks them but rarely bothers.

3.3 Confusion Patterns

To better understand the distribution, we found it useful to show the most frequent predicted labels among errors a la (Table 3). Note that five classes account for the bulk of the false positives.

Table 3: Top over-predicted classes among errors ("magnet" classes).

Predicted (in error)	**Count**
Rhipidura leucophrys	13
Rhipidura albiscapa	5
Colluricincla harmonica	5
Philemon citreogularis	5
Acanthorhynchus tenuirostris	5

Rhipidura leucophrys alone accounts for 13 of the roughly 90 errors observed across the test set, or approximately 14% of all misclassifications. The top five over-predicted classes between them account for around 37% of total errors. Notably, *Rhipidura leucophrys* is also the largest class in the training dataset, with 649 files, compared to 4 files for the smallest. This feels like a classic case of a class-imbalance-driven model defaulting to common training labels when uncertain about an input.

An asymmetry check (Table 4) confirms this is not a two-way street. For each of the most frequent confused pairs, the reverse direction occurs rarely or not at all.

Table 4: Most asymmetric confusions in the test set.

Actually was	**Called instead**	**A→B**	**B→A**
Acanthorhynchus tenuirostris	*Rhipidura leucophrys*	2	1
Anhinga novaehollandiae	*Colluricincla harmonica*	2	0
Climacteris picumnus	*Acanthorhynchus tenuirostris*	2	0
Fulica atra	*Manorina melanophrys*	2	0
Vanellus miles	*Rhipidura leucophrys*	2	0

If the model were genuinely confusing acoustically similar species, we would expect these confusions to be roughly symmetric, with A misclassified as B and B misclassified as A in similar proportions. They are not. Errors flow predominantly into the top classes from many distinct source classes,

with basically no flow in the opposite direction. It's a small flow, to be fair, but it's definitely something interesting about our model's behaviour.

3.4 Confidence Behaviour

Mean prediction confidence on correct predictions is 0.911, versus 0.448 on incorrect predictions; medians are 0.971 and 0.429 respectively. This is a nice clean separation, suggesting that the model's confidence output is a meaningful signal of overall correctness. Expected calibration error is 0.066, suggesting a gentle tendency toward overconfidence.

The practical consequence of this separation is that a confidence threshold can usefully filter the model's output. Table 5 sweeps thresholds from 0.50 to 0.95 against the test set.

Table 5: Precision-versus-share kept at various confidence thresholds.

Threshold	Share kept	Precision
0.50	0.938	0.985
0.60	0.914	0.991
0.70	0.890	0.993
0.80	0.839	0.995
0.85	0.800	0.995
0.90	0.733	0.996
0.95	0.596	0.997

From previous work we know that the HMI currently displays predictions above a confidence of 0.70, where precision is 0.993. Raising this to 0.80 improves displayed precision to 0.995 whilst suppressing only an additional 5% of predictions. Beyond 0.85, returns diminish sharply.

3.5 Secondary Slices

Performance was sliced by file format and class support to check for systematic effects. Accuracy varied modestly across formats (MP3 95.6%, OGG 93.0%, WAV 98.3%), though sample sizes for the non-MP3 formats are small enough that this variation should not be over-interpreted. Across class support bins, top-1 accuracy moved from 95.0% on classes with 10–20 training files to 96.7% on classes with 200+ files, a narrow 1.7-point spread. Mean confidence climbed more cleanly, from 0.81 on small classes to 0.94 on large ones. Neither slice produced a finding strong enough to recommend a remedial action on its own.

4 Discussion

4.1 The Dominant Failure Mode

The most actionable finding in this analysis involved the almost magnetic effect certain classes are having over the result. A small number of classes are over-predicted by the model when it is uncertain, dragging down precision on those classes whilst inflating false-positive counts across the rest of the dataset. The asymmetry analysis rules out bidirectional acoustic similarity as the cause; this is a class-imbalance artefact, with the model defaulting to the labels it saw most often during training.

Two options are reasonable recommendations for Sprint 3: a retrained model using class-weighted cross-entropy loss, or a post-training prior-correction step applied to the model's logits before softmax. The latter is a low-effort, training-free intervention that could recover a substantial fraction of the magnet-attributable errors; the former is more involved but offers a more thorough fix.

4.2 Near-Misses Versus Genuine Misses

That 61% of errors place the correct label within the top-5 is, in practice, a positive finding. It suggests that the model often recognises that something is acoustically present even when it gets the specific class wrong. This opens a UI-side lever for the HMI; showing the top 2 or top 3 predictions for low-confidence cases would let a human reviewer recover many of these errors without retraining anything. The remaining errors represent the model's genuine blind spots. Manual review of those specific clips would be a small, contained piece of work, and would clarify whether they are dominated by multi-species recordings, noise, label errors, or genuinely novel acoustic patterns.

5 Recommendations for Sprint 3

5.1 Raise the HMI Confidence Threshold to 0.80

The HMI currently filters predictions at a confidence of 0.70, where displayed precision is 0.993. Raising the threshold to 0.80 lifts precision to 0.995 whilst suppressing only an additional 5% of predictions; beyond 0.85, returns drop off sharply. This is a configuration change with no model-side implications and can be deployed immediately.

5.2 Apply Post-Hoc Prior Correction

The most actionable model-side finding is that around 37% of errors flow into five magnet classes, led by *Rhipidura leucophrys*. This is a class-imbalance artefact rather than acoustic confusion. Post-hoc prior correction, subtracting the log of training class priors from the model's logits before softmax, targets this directly without retraining.

5.3 Display Top-K Predictions for Borderline Cases

Of the 91 errors observed, 55 (61%) have the correct label within the model's top-5, and 35 of those at rank 2. In other words, when the model gets top-1 wrong, it has usually still recognised the correct species as a plausible candidate. Showing the top-2 or top-3 in the HMI when top-1 confidence is low would surface those candidates to human reviewers, recovering a substantial fraction of currently-misclassified inputs at the display layer alone.

5.4 Address Class Imbalance in the Training Data

The training distribution spans a massive range of samples per class. We go from 649 files in the largest class to 4 in the smallest, with this imbalance being the root cause of the magnet effect described previously Prior correction mitigates the symptoms at inference time, but the more durable fix is at the data layer: targeted collection for the under-represented classes flagged in Section 3.2, particularly those with high precision but low recall, where the model is accurate when it picks them but rarely does. Even 20–30 additional samples per under-represented class should measurably shift recall without any retraining changes beyond the additional data.

5.5 Further Recommendations

Temperature scaling on a held-out validation set could address the mild overconfidence. A scaled spot-check between the PyTorch and TFLite variants would verify the numerical equivalence assumed throughout this analysis. Once the magnet effect is addressed, retraining with class-weighted loss is a more permanent alternative to prior correction.